Lua — Reference

A **Lua** subset compiled with `lex` + `yacc` + `cc` (`lua.l` / `lua.y` lower Lua to C; `cc` lowers the C to .NET IL). Lua is small, dynamically typed, and built around one data structure — the **table** — and first-class **functions**. This implementation keeps that character: every value is a boxed `Val` (nil/boolean/number/string/table/function), tables serve as both arrays and hash maps, and each function body is lambda-lifted to a top-level .NET method reached through a numeric id, so functions can be stored in tables, passed around, and called from C#/VB.NET (Activity 9).

```
lua prog.lua                # compile to prog.exe (native .NET executable)
lua prog.lua -o app.exe     # choose the output name
lua lib.lua --dll           # compile to a library for C#/VB.NET
```

Quick Reference

```
-- comment
local x = 10                        -- local variable (global if no 'local')
local t = {1, 2, 3}                 -- table as an array (1-based)
local p = {name = "Ada", year = 1983}   -- table as a record; p.name, p["name"]
print(#t, t[1], p.name)             -- # = length; indexing with [] or .
x = x + 1                           -- arithmetic + - * / % ^
local s = "a" .. "b"                -- .. concatenates

if c then ... elseif d then ... else ... end
while c do ... end
repeat ... until c
for i = 1, 10 do ... end            -- numeric for (optional step: 1, 10, 2)
for k, v in pairs(t) do ... end     -- iterate a table
for i, v in ipairs(t) do ... end

local function fib(n)               -- functions are values; recursion is fine
  if n < 2 then return n end
  return fib(n - 1) + fib(n - 2)
end

local obj = {value = 0}             -- table-based objects
function obj:add(n) self.value = self.value + n end   -- : adds an implicit self
obj:add(5)                          -- method-call sugar passes obj as self
```

Data types

Eight... here six: **nil**, **boolean** (`true` / `false`), **number** (a double), **string**, **table**, and **function**. Variables are dynamically typed — any variable can hold any value. `nil` and `false` are the only *falsey* values; everything else (including `0` and `""`) is truthy.

The table

The table is Lua's only data structure and does the work of arrays, records, and dictionaries at once. `{1, 2, 3}` fills integer keys `1, 2, 3`; `{x = 1}` is `["x"] = 1`; `[expr] = v` keys by an arbitrary value. Index with `t[k]` or, for string keys, the sugar `t.k`. `#t` gives the array length. Objects are just tables whose fields happen to be functions.

Statements / Commands

Assignment (including **multiple assignment**, `a, b = b, a`), `if/elseif/else/end`, `while`, `repeat/until`, numeric `for i = a, b[, step]`, generic `for k, v in pairs(t)`, `return`, `break`, local declarations, and function definitions.

Functions

Functions are **first-class values**. Define them with `function name( ... ) ... end`, `local function name( ... )`, or anonymously as `function( ... ) ... end` assigned to a variable or table field. `return` yields a value; `:` in a definition or call adds the implicit `self` parameter (Lua's object sugar). Each function compiles to a top-level .NET method dispatched by id.

Input / Output

`print( ... )` writes its arguments separated by tabs and a trailing newline. `tostring(v)` gives a value's text; `type(v)` gives its type name; `tonumber(s)` parses a number.

Graphics

None. Lua's niche is embedding and scripting; Activity 8 draws a text histogram from a table of counts.

Notes

- Tables and arrays are **1-based**.
- `..` is string concatenation (numbers are coerced to text); `~=` is "not equal".
- `and` / `or` are short-circuit and return one of their operands (`a or b` is `a` if `a` is truthy, else `b`) — the idiom for defaults.
- `local` matters: an unqualified assignment creates/updates a **global**.

Subset boundaries

A faithful procedural + table core. Not included: **closures that capture an enclosing function's locals** (functions see globals and their own locals/params, not outer locals — named functions are treated as globals so recursion works); **multiple return values** (functions return one value; `pairs` / `ipairs` are handled specially by the `for` loop); **metatables** and metamethods (so no operator overloading or `__index` inheritance); **varargs** (`...`); **goto**; integer/float distinction (all numbers are doubles); and most of the standard library beyond `print` / `type` / `tostring` / `tonumber` / `pairs` / `ipairs` and the `#` operator. Function calls take up to four arguments.

Tutorial

Every example was compiled and run with `lua`; the output shown is real.

1. Your first program

```
print("hello from Lua")
```

```
hello from Lua
```

2. Variables and data types

```
local x = 6
print(x * 7)
print(3 + 4 * 2)
print(2 ^ 10)
```

```
42
11
1024
```

Variables are dynamically typed and declared with `local`. `3 + 4 * 2` is `11` — unlike some of our other languages, Lua *does* give `*` higher precedence than `+`.

3. Flow control

```
local sum = 0
for i = 1, 10 do sum = sum + i end
print(sum)

local i = 1
while i < 100 do i = i * 2 end
print(i)
```

```
55
128
```

`for i = 1, 10` counts inclusively; `while` repeats until its condition is falsey.

4. Tables

The one structure, used as array and record:

```
local t = {10, 20, 30}
print(#t, t[1], t[3])
t[4] = 40
print(#t)

local p = {name = "Ada", year = 1983}
print(p.name, p.year)

for i, c in ipairs({"red", "green", "blue"}) do print(i, c) end
```

```
3   10  30
4
Ada 1983
1   red
2   green
3   blue
```

`#t` is the array length; `p.name` is sugar for `p["name"]`; `ipairs` walks the array part in order.

5. Functions

Functions are values and may recurse:

```
local function fact(n)
  if n ≤ 1 then return 1 end
  return n * fact(n - 1)
end
print(fact(5))

local a, b = 1, 2
a, b = b, a                 -- multiple assignment swaps them
print(a, b)
```

```
120
2   1
```

6. Memory management

Values live on the managed **.NET heap** and are reclaimed by the garbage collector:

- Tables, strings, and numbers are boxed objects created as needed.
- There is no `free`; unreachable values are collected automatically by the CLR.
- This is exactly Lua's model — Lua is a garbage-collected language.

7. Strings

`..` concatenates (coercing numbers), `#` measures:

```
local s = "Lua"
print(#s)
print(s .. "!")
print("n = " .. 42)
```

```
3
Lua!
n = 42
```

8. Drawing a picture — a text histogram

```
local data = {3, 6, 2, 5}
for i = 1, #data do
  local bar = ""
  for j = 1, data[i] do bar = bar .. "#" end
  print(bar)
end
```

```
###
######
##
#####
```

A table of counts, an outer loop per row, and an inner loop building a bar of `#` by concatenation.

9. Objects with tables and `self`

Lua's object orientation is just tables plus the `:` sugar, which passes the receiver as `self`:

```
local acc = {balance = 100}
function acc.deposit(self, n) self.balance = self.balance + n end
function acc:withdraw(n) self.balance = self.balance - n end   -- : adds self
acc.deposit(acc, 50)
acc:withdraw(30)            -- sugar: acc is passed as self
print(acc.balance)
```

```
120
```

`acc:withdraw(30)` is exactly `acc.withdraw(acc, 30)`. (Full prototype inheritance needs metatables — see *Subset boundaries*.)

10. Calling Lua from C# / VB.NET

Compile a file of functions as a library. Because Lua is dynamically typed, C# interops at the value-runtime level: it runs the chunk once to register the global functions, looks each up by name, calls it with the call helpers, and unboxes results with `numval`.

```
-- lib.lua
function add(a, b) return a + b end
function fib(n)
  if n < 2 then return n end
  return fib(n - 1) + fib(n - 2)
end
```

```
lua lib.lua --dll          # → lualib.dll
```

```
using CRuntimeLib;                          // for CRuntime.InternString

CProgram.main(0, 0);                          // run the chunk: registers the globals
int add = CProgram.gget(CRuntime.InternString("add"));
int fib = CProgram.gget(CRuntime.InternString("fib"));
Console.WriteLine($"add(20, 22) = {CProgram.numval(CProgram.call2(add, CProgram.mknum(20), C
Console.WriteLine($"fib(15)     = {CProgram.numval(CProgram.call1(fib, CProgram.mknum(15)))}
```

```
add(20, 22) = 42
fib(15)     = 610
```

C# is calling real Lua functions. The interop is at the value level (boxed handles + `mknum` / `numval`) rather than via native signatures — the honest consequence of dynamic typing. From here the natural extensions are closures, multiple return values, and metatables (*Subset boundaries*).